

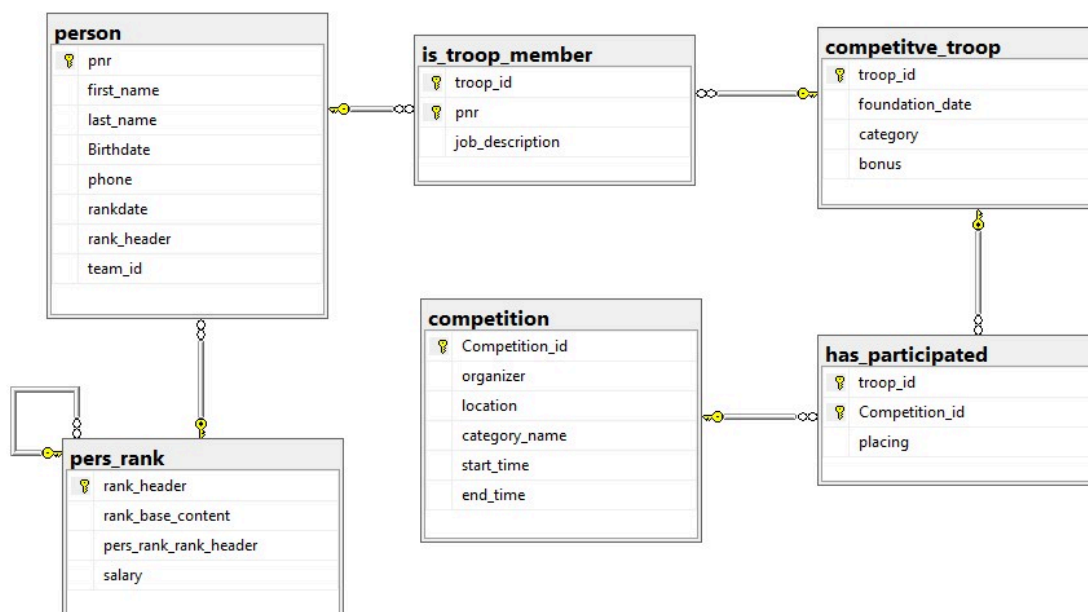
User Defined Functions and Stored Procedures

Allgemeines:

- Erstellen Sie eine Datenbank **Feuerwehr** und erstellen Sie die Daten mit Hilfe des Files Feuerwehr_create_table.sql
- Erstellen Sie ein Abgabefile **4Xhif_nachname.sql**, das Sie auf eduvidual hochladen. Ersetzen Sie das X mit Ihrer Klasse.
- Achten Sie auf das besprochene Styling, die Lauffähigkeit des Codes und es sollen alle Spalten einen Namen besitzen!
- Nummerieren Sie in Ihrem File die Aufgaben entsprechend der Angabe.

Das Datenbankmodell sieht folgendermaßen aus:

Hierbei handelt es sich um ein vereinfachtes Modell einer Feuerwehr, welches die wichtigsten Informationen über die Feuerwehrleute, ihre Dienstgrade, die Einsätze und deren Platzierungen bei Wettkämpfen enthält.



User Defined Functions:

1. Erstellen Sie eine Skalarwert Funktion `udf_bonus`, welche als Parameter die ID einer Person erhält.

```

create or alter function udf_bonus(@person_id int)
returns decimal(10,2)
as
.....
  
```

- Für erreichte Platzierungen in einem Wettbewerb erhalten die Wettkampftruppen-Mitglieder Bonuszahlungen.

- Im Falle eines Wettbewerbssieges werden 100% der für die Truppe festgelegten Bonuszahlung ausgeschüttet. Ein zweiter Platz bringt 50% der Bonuszahlung und ein dritter Platz immerhin noch 25%.
- Alle anderen Platzierungen erhalten keinen Bonus für die Absolvierung des Wettbewerbs.
- Die Funktion `udf_bonus` soll nun für eine Person, anhand der von ihr in Wettkampftruppen bestrittenen Wettkämpfe, den Gesamtbonus nach dem obigen Schema berechnen und zurückliefern.
- Hat eine Person gar keinen Wettkampf bestritten, oder wurde nie eine Platzierung innerhalb der ersten drei Ränge erreicht, soll der Wert 0.0 zurückgeliefert werden.

2. Erstellen Sie eine Tablevalued Funktion `udf_hierarchie_dienstgrad`, welche als Parameter einen bestimmten Dienstgrad erhält.

```
create or alter function udf_hierarchie_dienstgrad (@rank_header VARCHAR(30))
.....
```

- Es sollen diesem Dienstgrad übergeordneten Dienstgrade ausgegeben werden.
- Geben Sie für jeden Dienstgrad die ID, Beschreibung sowie das zugehörige Gehalt und den Gehaltsunterschied zum nächstniedrigen Dienstgrad aus.
- Sollte es keinen niedrigeren Dienstgrad geben, soll der Inhalt der Spalte 0 sein

Stored Procedures:

3. Erstellen Sie eine Prozedur `stp_erhoehe_dienstgrad`, welche als Parameter eine Zahl (Integer) erhält.

```
create or alter procedure stp_erhoehe_dienstgrad
    @jahre INT
AS
BEGIN
.....
```

- Diese Zahl beschreibt die Zeitspanne in Jahren, die vergehen muss, bevor eine Person zum nächsthöheren Dienstgrad aufsteigen kann.
- Betrachten Sie für alle Personen, wie lange sich diese Person bereits in ihrem Dienstgrad befindet (Differenz zwischen dem aktuellen Datum und dem gespeicherten Wert der jeweiligen Person).
- Ist dieser Wert größer oder gleich dem Wert des Parameters, dann steigt diese Person um einen Dienstgrad auf.
- Hat eine Person den höchstmöglichen Dienstgrad erreicht, dann verbleibt sie natürlich in diesem.
- Geben Sie die Vorname, Nachname und Bezeichnung des neuen als auch des alten Dienstgrades aus, bei all jenen Personen, die einen Dienstgrad aufgestiegen sind.
- Stellen Sie sicher, dass es sich beim Parameter der Prozedur um einen positiven Wert handelt, andernfalls brechen Sie die Verarbeitung mit einer passenden Fehlermeldung ab.